Name/NetID: Nathan Kariuki / nkari011

Session Date: April 29, 2026 (1-2 hours)

Objective: Set up the project and implement CSR graph loading (with upside and reverse edges).

Attempts made:

I created the basics of CsrGraph with the edges being forward. The first attempt had an issue, as I forgot a prefix sum step for maintaining the edge order. This was the first step that contributed to the edges being ordered:

cpp

g.offsets[i] += g.offsets[i-1];

I had to also dynamically calculate the vertex count, as it was not specified. I found another issue with the reverse CSR I created where I used the offsets array as a backward tracing temporary array. As a result the array was corrupted.

Solutions/ Workarounds:

The issue was resolved with the assumption that there are two temporary variables used for the tracers:

vector<uint32_t> cur_out = g.offsets;

vector<uint32_t> cur_in = g.in_offsets;

The same assumption was used for reverse CSR with src and dst swapped. I also had to implement a way to skip over comments, as the LiveJournal file has a number of these (i.e., #) which were causing the parser to break:

if (line.empty() || line[0] == '#') continue;

Commits: initial project setup with a basic graph structure; implemented CSR graph loading (forward edges); added reverse CSR (in-edges)

Experiments/ Analysis:

I tested and verified the result with a small graph of 5 vertices including a disconnected component. I observed that vertex 1 had 1 out-neighbor and 1 in-neighbor and vertices 3 and 4 were completely disconnected. This confirmed the hand calculations I had made.

Learning outcome:

I discovered that reverse CSR would load the edges into the totally incorrect positions without the prefix sum step. I realized that reverse CSR is important because the BSP pull model requires in-edges for every vertex which greatly improves the performance of the algorithm.